

Technische Internetbasierte Systeme

Auftakt, Ablauf und Projektarbeit

Linus Hoppe

Aalen University

31. März 2026





- ① Arduino-Grundlagen
- ② Digital Output
- ③ Digital Input
- ④ Analog Input
- ⑤ Analog Output
- ⑥ Hardware-Verbindungen



- ① Arduino-Grundlagen
- ② Digital Output
- ③ Digital Input
- ④ Analog Input
- ⑤ Analog Output
- ⑥ Hardware-Verbindungen



```
#include <Arduino.h>

void setup() {
  Serial.begin(115200);
}

void loop() {
  Serial.println("Loop running");
  delay(1000);
}
```

Ablauf

- Initialisierung in setup()
- Wiederholung in loop()

Ergebnis

- Ausgabe jede Sekunde



- Kommunikation über USB / UART
- Ausgabe mit:
 - `Serial.print(...)`
 - `Serial.println(...)`
- Verwendung:
 - Messwerte und Zustände sichtbar machen
 - Programmablauf nachvollziehen

Einschränkung

Kein Hardware-Debugger → keine Breakpoints oder Schritt-Ausführung

Grund

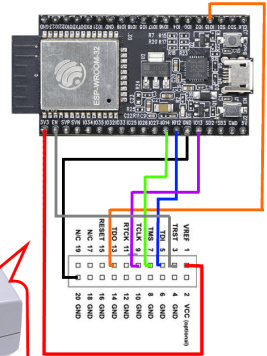
ESP8266-Boards (z. B. Wemos D1 mini) besitzen kein herausgeführtes JTAG-Interface

Hardware-Debugging (JTAG)



- Debugging mit externer Hardware möglich
- Beispiel: JTAG-Debugger (z. B. Segger J-Link)
- Funktionen:
 - Breakpoints setzen
 - Schrittweise Ausführung
 - Speicher und Register einsehen
- Voraussetzung:
 - JTAG-Schnittstelle muss verfügbar sein

ESP32	
Signal	IO Pin
TDI	IO12
TCK	IO13
TDO	IO15
TMS	IO14



Beispiel: JTAG-Debugger



- Der ESP8266 führt **mehr als nur dein Programm** aus:
 - WLAN-Verbindung
 - TCP/IP-Stack
- Diese Aufgaben laufen **im Hintergrund**
- Wichtig:
 - dein Code teilt sich die CPU mit dem System

Konsequenz

Das System benötigt regelmäßig Rechenzeit



```
void loop() {  
    readSensor();  
    processData();  
    sendData();  
  
    delay(10); // gives time to WiFi + system  
}
```

Prinzip

- kurze, schnelle Funktionen
- keine langen Schleifen
- regelmäßige CPU-Freigabe

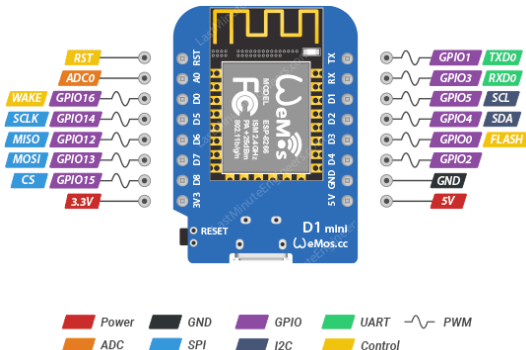
Technischer Hintergrund

Systemtasks laufen nur zwischen loop()-Durchläufen oder bei delay() bzw. yield()



- ① Arduino-Grundlagen
- ② Digital Output
- ③ Digital Input
- ④ Analoger Input
- ⑤ Analog Output
- ⑥ Hardware-Verbindungen

Pinout verstehen (Wemos D1 mini)



Was zeigt das Pinout?

- ein Pin hat oft mehrere Bezeichnungen
- z. B. D1 und GPIO5
- manche Pins haben Zusatzfunktionen

Wichtig für die Praxis

- nicht jeder Pin ist beliebig nutzbar
- bei Bibliotheken und Funktionen auf den Pin achten



```
// bevorzugt: direkte GPIO-Nutzung
const int LED_PIN = 5;      // GPIO5 (D1)
const int BUTTON_PIN = 4;   // GPIO4 (D2)

// alternative: Board-Definition
#define LED_PIN_D D1
#define BUTTON_PIN_D D2

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT);
}
```

Varianten

- `const int NAME = GPIO`
 - echte Hardware-Referenz
- `#define NAME D1`
 - Board-Abstraktion

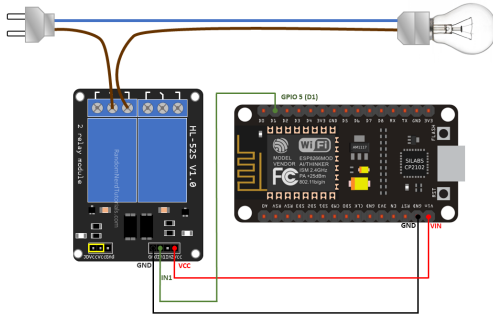
Einordnung

- GPIO → präzise, datenblattnah
- D-Notation → einfacher beim Verkabeln

Zuordnung

D1 = GPIO5, D2 = GPIO4

Beispiel: Relais / Last schalten



- GPIO steuert Relais
- Relais schaltet externe Last (z. B. Lampe)
- häufig: LOW → aktiv

Prinzip

- `pinMode(pin, OUTPUT)`
- `digitalWrite(pin, HIGH / LOW)`

Elektrisch

- HIGH → ca. 3.3V
- LOW → 0V



```
#define LED_PIN D1

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);

  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

Wichtige Funktionen

- pinMode(pin, OUTPUT)
- digitalWrite(pin, HIGH)
- digitalWrite(pin, LOW)

Frage

- Was passiert hier?



```
#define LED_PIN D1

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);

  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

Wichtige Funktionen

- pinMode(pin, OUTPUT)
- digitalWrite(pin, HIGH)
- digitalWrite(pin, LOW)

Wirkung

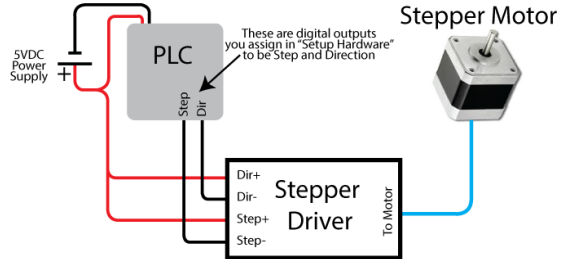
Die LED blinkt im 500-ms-Takt

Steuerung

- DIR (Richtung)
 - HIGH / LOW
- STEP (Impulse)
 - jeder Puls = ein Schritt

Bedeutung

- Frequenz → Geschwindigkeit
- Anzahl Pulse → Position

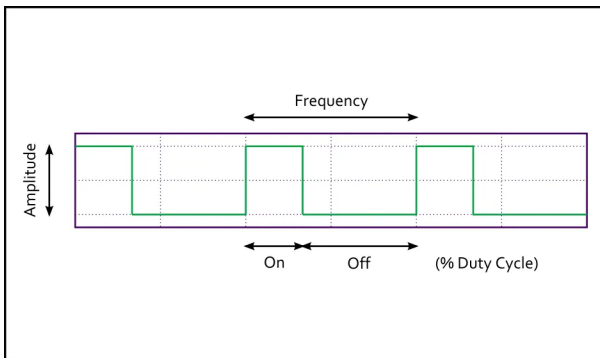


Prinzip

- Signal besteht aus Impulsen
- jeder Impuls = ein Ereignis

Bedeutung

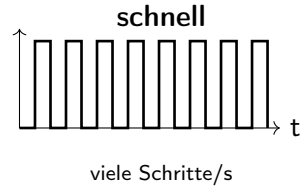
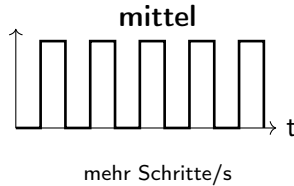
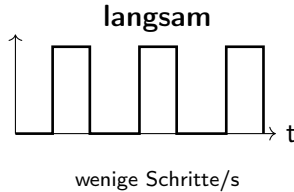
- Frequenz bestimmt:
 - wie oft ein Ereignis passiert



Beispiel

Schrittmotor: jeder Puls am STEP-Eingang entspricht einem Schritt

STEP-Signal: Frequenz bestimmt Geschwindigkeit



Interpretation

Gleiche HIGH- und LOW-Zeit, aber kürzere Gesamtperiode → höhere Frequenz → höhere Schrittgeschwindigkeit



```
const int stepPin = 5;
const int dirPin = 4;
bool direction = true;

void setup() {
  pinMode(stepPin, OUTPUT);
  pinMode(dirPin, OUTPUT);
}

void loop() {
  digitalWrite(dirPin, direction);

  for (int i = 0; i < 100; i++) {
    digitalWrite(stepPin, HIGH);
    delayMicroseconds(500);
    digitalWrite(stepPin, LOW);
    delayMicroseconds(500);
  }

  direction = !direction;
  delay(500);
}
```

Signale

- DIR
- STEP

Frage

- Was passiert hier?
- Wie bewegt sich der Motor?



```
const int stepPin = 5;
const int dirPin = 4;
bool direction = true;

void setup() {
  pinMode(stepPin, OUTPUT);
  pinMode(dirPin, OUTPUT);
}

void loop() {
  digitalWrite(dirPin, direction);

  for (int i = 0; i < 100; i++) {
    digitalWrite(stepPin, HIGH);
    delayMicroseconds(500);
    digitalWrite(stepPin, LOW);
    delayMicroseconds(500);
  }

  direction = !direction;
  delay(500);
}
```

Signale

- DIR → Richtung
- STEP → Impulse

Ablauf

- 100 Schritte vorwärts
- Richtungswechsel
- 100 Schritte zurück

Timing

- 500 μ s HIGH + LOW
- Frequenz → Geschwindigkeit

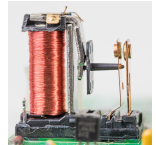
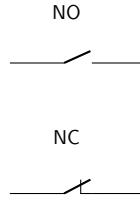


- ① Arduino-Grundlagen
- ② Digital Output
- ③ Digital Input**
- ④ Analog Input
- ⑤ Analog Output
- ⑥ Hardware-Verbindungen

Taster: Grundprinzip



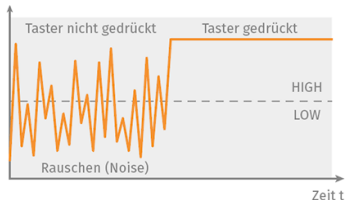
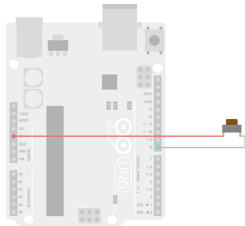
- Ein Taster verändert einen Stromkreis
- offen → kein Stromfluss
- gedrückt → Kontakt geschlossen
- digitales Signal:
• **HIGH** oder **LOW**



Pull-Up und Pull-Down



- Problem:
 - Eingang kann „floaten“
 - Störungen → zufällige Werte
- Lösung:
 - Widerstand definiert Ruhezustand
- Varianten:
 - Pull-Up:
 - Eingang standardmäßig HIGH
 - Pull-Down:
 - Eingang standardmäßig LOW

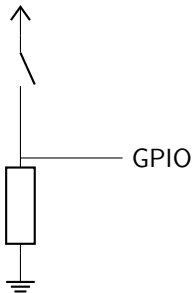


Grund

Vermeidung von Rauschen und undefinierten Pegeln



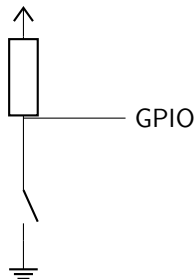
- Widerstand zieht Eingang auf **LOW**
- Verhalten:
 - Taster offen → LOW
 - Taster gedrückt → HIGH
- Funktion:
 - verhindert „floating“
 - definiert Ruhezustand



Merke

Pull-Down bedeutet: Ruhezustand LOW, Tastendruck HIGH

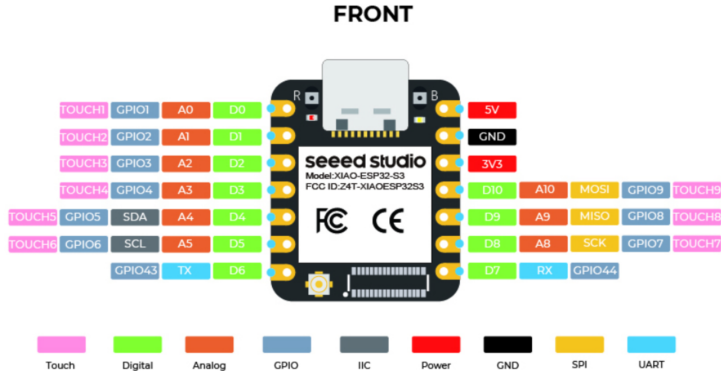
- Widerstand zieht Eingang auf **HIGH**
- Verhalten:
 - Taster offen → HIGH
 - Taster gedrückt → LOW
- Praxis:
 - oft intern im Mikrocontroller vorhanden
 - Standard bei Arduino / ESP8266



Merke

Pull-Up bedeutet: Ruhezustand HIGH, Tastendruck LOW

Pins am XIAO ESP32-S3



- Für den Taster:
 - freien digitalen Pin wählen
 - Pins mit Sonderfunktion vermeiden
- Praxis:
 - interner Pull-Up verfügbar



```
#define BUTTON_PIN D2

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(115200);
}

void loop() {
  int state = digitalRead(BUTTON_PIN);

  Serial.println(state);
  delay(100);
}
```

Wichtige Funktionen

- `pinMode(pin, mode)`
INPUT,
OUTPUT,
INPUT_PULLUP
- `digitalRead(pin)`
liest HIGH / LOW
- `digitalWrite(pin, value)`
setzt HIGH / LOW

Logik

INPUT_PULLUP: gedrückt → LOW, offen → HIGH



- Alternative zu **Polling**:
 - kein permanentes Abfragen notwendig
 - Reaktion erfolgt ereignisgesteuert
- Prinzip:
 - Signaländerung an einem Pin löst Funktion aus
- Signalzustände:
 - LOW $\rightarrow$ 0V
 - HIGH $\rightarrow$ logische 1 (z. B. 3.3V)
- Trigger-Modi:
 - RISING $\rightarrow$ LOW $\rightarrow$ HIGH
 - FALLING $\rightarrow$ HIGH $\rightarrow$ LOW
 - CHANGE $\rightarrow$ jede Änderung
 - HIGH, LOW $\rightarrow$ levelbasiert

Einordnung

Level-Trigger (HIGH/LOW) sind selten sinnvoll

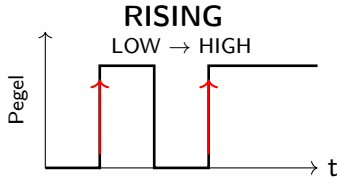
Problem:

- wird mehrfach ausgelöst
- kein klares Ereignis

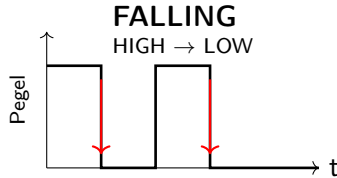
Praxis:

- meist Flanken verwenden:
- RISING, FALLING

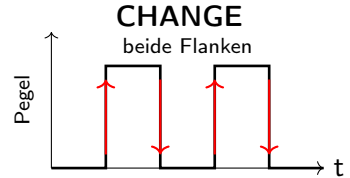
Interrupt-Trigger (Flanken)



- steigende Flanken



- fallende Flanken



- jede Änderung



```
#define BUTTON_PIN D2

volatile int counter = 0;
volatile bool flag = false;

void IRAM_ATTR onPress() {
    counter++;
    flag = true;
}

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    attachInterrupt(
        digitalPinToInterrupt(BUTTON_PIN),
        onPress,
        FALLING
    );
}

void loop() {
    if (flag) {
        flag = false;
        Serial.println(counter);
    }
}
```

IRAM_ATTR

- Funktion liegt im **RAM**, nicht im Flash
- notwendig beim ESP8266/ESP32
- Interrupts müssen **sofort** ausführbar sein

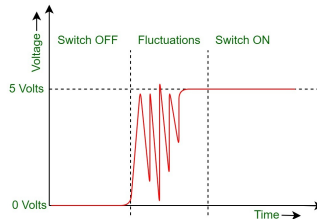
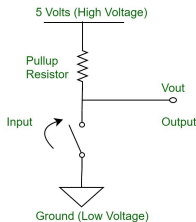
volatile

- Variable kann sich **außerhalb des normalen Programmflusses** ändern
- Zugriff aus ISR und loop()
- verhindert Optimierungen des Compilers

Kontaktprellen (Switch Bounce)



- Mechanische Kontakte schalten nicht ideal
- mehrere schnelle Übergänge (Prellen)
- Folge:
 - mehrere Flanken
 - mehrere Interrupts
- Problem:
 - ein Tastendruck → mehrere Events



Video

Kontaktprellen:

<https://www.youtube.com/watch?v=rqb53mV7GWc>



```
volatile unsigned long last = 0;
volatile int count = 0;

void IRAM_ATTR isr() {
    unsigned long now = millis();

    if (now - last > 50) {
        count++;
        last = now;
    }
}
```

Prinzip

- Zeitfenster: 50 ms
- weitere Flanken werden ignoriert

Wirkung

- genau ein Event pro Tastendruck



- ① Arduino-Grundlagen
- ② Digital Output
- ③ Digital Input
- ④ Analog Input
- ⑤ Analog Output
- ⑥ Hardware-Verbindungen

Analoger Eingang: Potentiometer



Prinzip

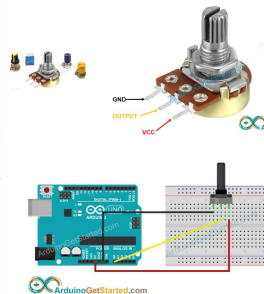
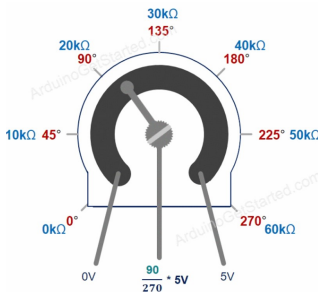
- verstellbarer Widerstand
- bildet einen Spannungsteiler

Anschluss

- 3 Pins:
 - 3.3V
 - GND
 - Mittelabgriff → A0

Funktion

- Drehung verändert Spannung
- Messung mit: `analogRead()`



Analoger Eingang: Potentiometer



Prinzip

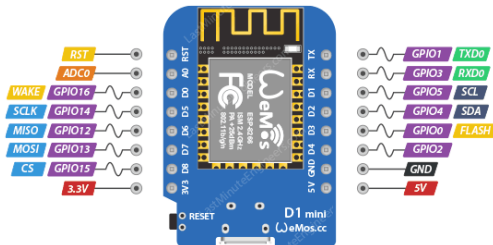
- verstellbarer Widerstand
- Spannungsteiler

Anschluss

- 3.3V, GND
- Mittelabgriff → A0

Messung

- `analogRead(A0)`
- liefert Wert statt HIGH / LOW



Hinweis

`analogRead()` nicht zu häufig aufrufen → sonst Probleme mit WLAN (mind. ca. 5 ms Abstand)

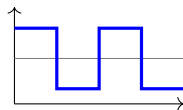


Prinzip

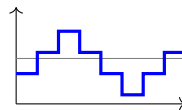
- analoges Signal ist kontinuierlich
- ADC bildet daraus eine digitale Zahl
- mehr Bit $\rightarrow$ mehr Stufen

Auflösung

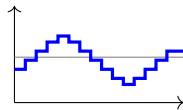
- 1 Bit $\rightarrow$ 2 Stufen
- 2 Bit $\rightarrow$ 4 Stufen
- 4 Bit $\rightarrow$ 16 Stufen
- 10 Bit $\rightarrow$ 1024 Stufen



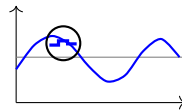
1-bit



2-bit



4-bit



16-bit

Grobe Funktionsidee

Ein ADC vergleicht das Eingangssignal mit internen Referenzwerten. Bei SAR-ADCs geschieht das schrittweise über Vergleich, Schalter und ein internes Widerstands- bzw. DAC-Netzwerk.



```
const int potPin = A0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int value = analogRead(potPin);

  float voltage = value * (3.3 / 1023.0);

  Serial.print("Voltage: ");
  Serial.println(voltage);

  delay(200);
}
```

Messwerte

- ADC:
 - 10 Bit $\rightarrow$ 0 ... 1023
- Spannung:
 - 0 ... 3.3 V

Umrechnung

$$U = \frac{\text{value}}{1023} \cdot 3.3 \text{ V}$$



- kein `pinMode(..., INPUT)` notwendig
- gilt für analoge Pins

Grund

`analogRead()` übernimmt die Konfiguration automatisch

Zweck

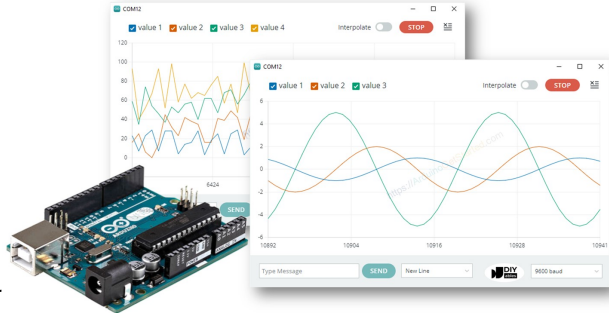
- Visualisierung von Messwerten
- Verlauf statt Einzelwerte

Voraussetzung

- `Serial.println(value);`

IDE

- Werkzeuge → Serieller Plotter





Mechanisch

- Potentiometer (Position)
- Dehnungsmessstreifen (Kraft)

Licht

- Photodiode / LDR

Temperatur

- NTC / PTC

Umgebung

- Feuchtigkeitssensor



- ① Arduino-Grundlagen
- ② Digital Output
- ③ Digital Input
- ④ Analog Input
- ⑤ Analog Output**
- ⑥ Hardware-Verbindungen



- Ausgang bleibt digital:
 - nur 0 V oder 3.3 V
- „Analoger“ Effekt entsteht durch:
 - Trägheit von Systemen

Beispiele

- LED → erscheint gedimmt
- DC Motor → läuft langsamer

Interpretation

System mittelt das schnelle Ein/Aus-Signal

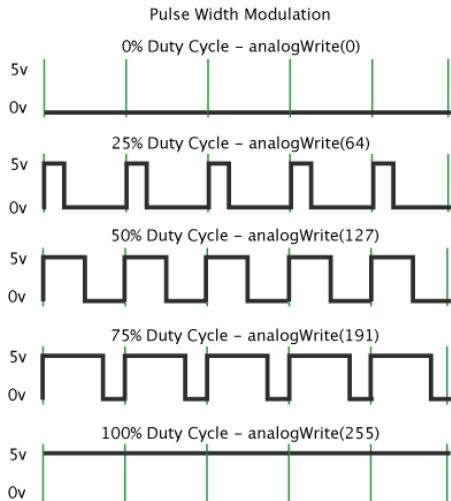


PWM

- schnelles Umschalten zwischen HIGH und LOW
- Duty Cycle = Anteil der HIGH-Zeit

Wichtig

- kein echter Analogwert
- nur zeitlicher Mittelwert





- `analogWrite(pin, value)` erzeugt PWM
- Ausgang bleibt ein digitaler Pin
- I/O-Pegel des Boards:
 - **0 V** oder **3.3 V**
- Standard:
 - Wertebereich: 0 ... 255
 - PWM-Frequenz: 1 kHz

Merke

Am Pin liegt nicht direkt eine konstante analoge Spannung an

PWM: Interpretation eines Wertes



Gegeben

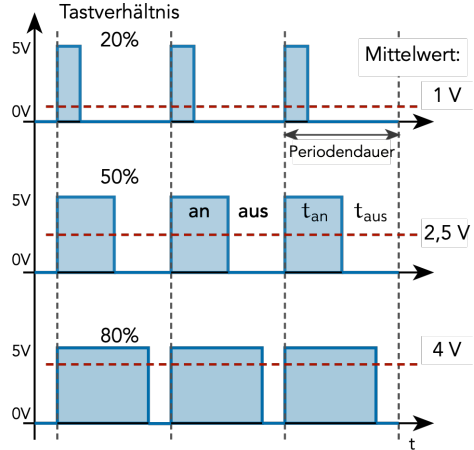
- `analogWrite(pin, 127)`
- Versorgung: 3.3 V

Fragen

- Welche mittlere Spannung entspricht diesem Wert?

Zusatz

- Welche Spannung liegt tatsächlich am Pin an?



Duty Cycle

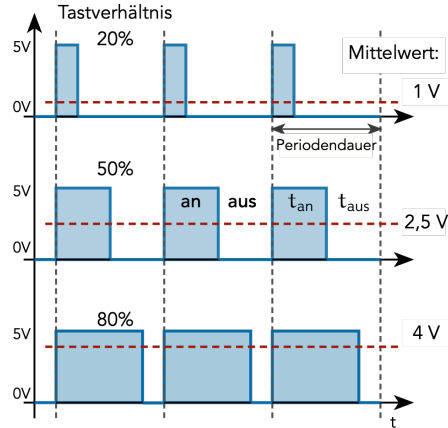
- $\frac{127}{255} \approx 50\%$

Mittlere Spannung

- $U \approx 3.3 \text{ V} \cdot 0.5$
- $\Rightarrow \text{ca. } 1.65 \text{ V}$

Tatsächliches Signal

- nur:
 - 0 V
 - 3.3 V
- kein konstanter Analogwert



Interpretation

Der Mittelwert entsteht nur durch zeitliche Mittelung (z. B. LED, Motor, Filter)



- ① Arduino-Grundlagen
- ② Digital Output
- ③ Digital Input
- ④ Analog Input
- ⑤ Analog Output
- ⑥ Hardware-Verbindungen

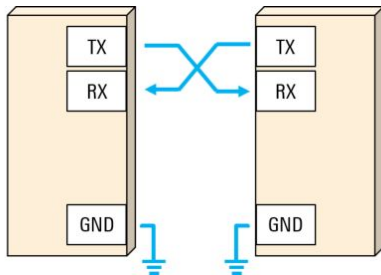


- Grundverbindung:
 - TX RX
 - gemeinsame Masse (GND)

- Initialisierung:

```
Serial.begin(115200);
```

- Typische Baudraten (Bit/s):
 - 9600
 - 115200



Varianten

- UART (TTL-Pegel)
- RS232 (klassisch, $\pm 12V$)
- RS485 / Modbus (robust, Bus-System)

Serielle Übertragung (UART)



- Übertragung:
 - Bit für Bit (asynchron)
- Ablauf eines Frames:
 - **Startbit** (Beginn)
 - **Datenbits** (z. B. 8 Bit)
 - **Parität** (optional)
 - **Stopbit** (Ende)
- Parität (Beispiel):
 - gerade Parität:
 - Anzahl der 1-Bits wird **gerade**

